C2M Coherence Clusters, JMS Queues & Inbound Web Services (IWS) Configuration

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — C2M Coherence, JMS & Inbound Web Services Configuration 4

The problem . . . 4

What this solution does . . . 4

How it's delivered . . . 4

The business outcome . . . 4

Who it's for . . . 4

Solution Overview — C2M Runtime: Coherence, JMS & Inbound Web Services 5

What "the C2M runtime" means here . . . 5

How the layers support batch, real-time, and integration . . . 6

Deployment model . . . 6

Version note . . . 6

Sources . . . 6

Requirements — C2M Coherence, JMS & IWS Configuration 7

1. A running WebLogic domain hosting the C2M/OUAF managed servers . . . 7

2. Managed-server / cluster topology decision (single vs. clustered) . . . 7

3. Network & ports for Coherence discovery . . . 7

4. Persistent store location for JMS (file or JDBC) . . . 8

5. C2M message senders / JMS integration requirements defined . . . 8

6. IWS enabled and the web-services EAR deployable . . . 8

7. Inventory of inbound services to enable . . . 9

8. Backups and a rollback point before changes . . . 9

Global Requirements — Oracle Utilities C2M / WebLogic Solutions 10

1. Licensed Oracle Utilities C2M + WebLogic entitlement . . . 10

2. Certified platform stack (OS, Java, Database, WebLogic) . . . 10

3. Environment topology (Dev / Test / Prod) & access . . . 10

4. Administrative access & tooling (WLST, shell, SQL) . . . 11

5. Change control, backup & a non-production pilot . . . 11

6. Collaborating teams & support model . . . 11

Implementation Guide — Configuring the C2M Runtime (Coherence · JMS · IWS) 12

Phase 0 — Assess (read-only) . . . 12

Phase 1 — Configure the Coherence cluster . . . 12

Phase 2 — Configure JMS (server → module → destinations → connection factory) . . . 13

Phase 3 — Deploy & enable Inbound Web Services (IWS) . . . 14

Phase 4 — Validate the runtime end to end (Pilot) . . . 15

Phase 5 — Expand to remaining environments . . . 15

Rollback . . . 16
Sources . . . 16
Testing — Validating Coherence, JMS & IWS 16
A. Coherence clustering . . . 16
B. JMS messaging . . . 17
C. Inbound Web Services (IWS) . . . 18
D. Consolidated runtime health . . . 19
Troubleshooting — Coherence, JMS & IWS 19
Coherence . . . 19
JMS . . . 20
Inbound Web Services (IWS) . . . 20
Escalation . . . 21
Sources . . . 21
Best Practices — Coherence, JMS & IWS on C2M 22
Coherence . . . 22
JMS . . . 22
Inbound Web Services (IWS) . . . 23
Cross-cutting . . . 23
Sources . . . 24
Deliverables Summary — C2M Coherence, JMS & IWS Configuration 24
Documentation . . . 24
Automation scripts (`scripts/`) . . . 25
How they work together . . . 25
Outcome . . . 26
Appendix A — PowerShell Scripts 27

Executive Summary — C2M Coherence, JMS & Inbound Web Services Configuration

The problem

Oracle Utilities **Customer to Meter (C2M)** only clusters, messages, and integrates when three runtime layers are configured correctly on **Oracle WebLogic Server**: the **Coherence cluster**, the **JMS queues/topics and connection factories**, and the **Inbound Web Services (IWS)** that expose C2M to external systems. When any one is misconfigured, the symptoms are business-visible — the cluster won't form, messages back up or vanish, inbound integration calls fail — and the causes are easy to misdiagnose because the three layers depend on one another.

What this solution does

It configures the **entire C2M runtime layer as one coherent unit**, pilot-first:

- **Coherence** — a single named cluster the managed servers reliably join (unicast WKA, correct listen port), verified on a cold restart.
- **JMS** — a JMS server with a persistent store, a module with named queues/topics and a connection factory, targeted explicitly via a subdeployment, with JNDI names reconciled to C2M's message senders so messages actually flow.
- **IWS** — the `spl-web-services` application deployed with the repeatable OUAF `iwsdeploy` utility and only the required inbound services enabled, secured, and responding.

Delivered as **self-guided documentation plus five automation scripts** (WLST/Jython, shell, SQL — the authentic C2M/WebLogic toolchain) for discovery, validation, reporting, safe remediation, and scoping.

How it's delivered

The proven **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model: all changes are made on a **non-production environment first**, behind explicit pass/fail gates, with a documented rollback, before promoting the **same named objects and same scripts** to Test and Production.

The business outcome

The C2M runtime wired correctly — **Coherence clustering, JMS messaging, and IWS integration all healthy** — proven on a pilot before you scale, with saved evidence for change control. Self-guided and tested, at a fraction of the cost of a bespoke consulting engagement.

Who it's for

C2M/OUAF administrators, WebLogic middleware engineers, and integration teams standing up or repairing the C2M runtime on WebLogic 14c (reference stack C2M 25.4 / minimum v2.9; version-specific details are cited or flagged "verify for your version", and My-Oracle-Support-gated notes are pointed to, not reproduced).

Solution Overview — C2M Runtime: Coherence, JMS & Inbound Web Services

Oracle Utilities **Customer to Meter (C2M)** runs on the **Oracle Utilities Application Framework (OUAF)** deployed on **Oracle WebLogic Server**. Beyond the base install, three runtime layers determine whether C2M actually clusters, messages, and integrates in real time:

1. **Oracle Coherence clustering** — the in-memory data grid WebLogic/OUAF uses for cache coordination and (in clustered topologies) cross-node coherence. If the Coherence cluster does not form cleanly, managed servers behave as isolated nodes and cache-dependent behaviour is inconsistent.
1. **JMS queues, topics & connection factories** — the WebLogic JMS resources that carry asynchronous work (outbound message dispatch, real-time integration hand-offs, and any JMS consumers wired to OUAF). Misconfigured JMS servers, modules, or subdeployments cause messages to pile up, mis-target, or silently drop.
1. **Inbound Web Services (IWS)** — the OUAF mechanism (the replacement for the legacy **XAI** inbound services) that exposes C2M business services as SOAP/REST endpoints deployed as a Java EE web application on WebLogic. If IWS is not deployed and enabled, external systems cannot call into C2M.

This solution configures **all three runtime layers** as one coherent unit, because in practice they are interdependent: IWS rides on the same managed servers that participate in the Coherence cluster, and many real-time flows terminate in JMS. Getting one right while another is broken still leaves the platform unintegrated.

What "the C2M runtime" means here

Layer	WebLogic/OUAF object	What it enables	Symptom when wrong
Coherence	Coherence cluster + managed Coherence membership	Cache coordination, clustered scale-out	Cluster won't form; nodes act alone
JMS	JMS server, JMS system module, queues/topics, connection factory, subdeployment	Asynchronous messaging & real-time hand-off	Queues back up; messages mis-target
IWS	`spl-web-services` EAR deployed via `iwsdeploy` + enabled inbound services	Inbound SOAP/REST integration into C2M	External calls fail / 404

> ***Naming convention used throughout.*** *So that the docs, scripts, and console steps line up, this solution refers to a single* ***named set of objects*** *and reuses those names everywhere: the Coherence cluster* `c2m-coherence-cluster`*, the JMS server* `C2MJMSServer`*, the JMS system module* `C2MJMSModule`*, the connection factory* `C2MConnectionFactory` *(JNDI* `jms/C2MConnectionFactory`*), a representative queue* `C2MRealTimeQueue` *(JNDI* `jms/C2MRealTimeQueue`*), a representative topic* `C2MEventTopic` *(JNDI* `jms/C2MEventTopic`*),*

the subdeployment `C2MJMSSubDeployment`*, and the inbound web-service EAR* `spl-web-services`*. Substitute your own environment's standard names — but pick them once and reuse them, exactly as shown here.*

How the layers support batch, real-time, and integration

• **Batch** relies on a healthy domain and, in clustered deployments, on Coherence membership so that batch threadpools and cache behaviour are consistent across nodes.

• **Real-time** flows (online screens, service scripts) use IWS to receive inbound calls and JMS to hand off asynchronous work without blocking the caller.

• **Integration** with SOA/middleware and external systems is inbound via IWS and (frequently) asynchronous via JMS queues/topics — the two together are the C2M "integration surface".

Deployment model

Like every Oracle solution in this catalogue, work proceeds **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, on a **non-production environment first**, with explicit pass/fail gates and a documented rollback before any production change. The included automation is **WLST (Jython), shell, and SQL** — never PowerShell — because that is the authentic C2M/WebLogic toolchain.

Version note

• **C2M 25.4** (minimum **v2.9**) on **Oracle WebLogic Server 14c** is the reference stack; earlier releases differ (notably the XAI → IWS transition and Coherence packaging). Where a menu path, utility flag, or default differs by release, the step is flagged **"verify for your version"**.

• Some Coherence/IWS notes and patches are entitlement-gated on **My Oracle Support (MOS)** (for example, **Doc ID 1644914.1 — Migrating from XAI to IWS**); this solution points to them rather than reproducing gated content.

Sources

• Oracle® Fusion Middleware — *Configuring and Managing Coherence Clusters* (WebLogic Server 12.2.1.4 / 14.1.1): https://docs.oracle.com/en/middleware/fusion-middleware/weblogic-server/12.2.1.4/clust/coherence.html

• Oracle® Fusion Middleware — *Using WLST to Manage JMS Servers and JMS System Module Resources* (WebLogic 14.1.1): https://docs.oracle.com/en/middleware/standalone/weblogic-server/14.1.1.0/jmsad/wlst.html

• Oracle Utilities Application Framework — *Server Administration Guide* (Web Application Server Configuration; IWS): https://docs.oracle.com/en/industries/energy-water/framework/254/server-admin-guide/OUAF_SERVER_ADMIN_254/OUAF_Server_Admin_Configuration.06.4.html

• Oracle Utilities CCB — *Deploying Inbound Web Services (IWS)* (installation guide):

https://docs.oracle.com/en/industries/energy-water/ccb/29011/ccb-installation-guide/CCB-INSTALLATION-GUIDE/CCB_Install_Postinstall.09.09.html

• My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Requirements — C2M Coherence, JMS & IWS Configuration

These are the **solution-specific** prerequisites for configuring the C2M runtime layer. They are in addition to the **Global Requirements** for every Oracle solution (licensed C2M + WebLogic entitlement, certified platform stack, Dev/Test/Prod topology, WLST/shell/SQL tooling, change control + backup + non-prod pilot, and collaborating teams) — verify those first.

Each requirement below states **Description**, **Why it's required**, **How to verify**, and **How to implement if missing**.

1. A running WebLogic domain hosting the C2M/OUAF managed servers

• **Description:** An installed and startable OUAF/C2M WebLogic domain with the Administration Server and at least one managed server hosting the C2M application.

• **Why required:** Coherence membership, JMS servers, and the IWS EAR are all domain-scoped WebLogic objects — there is nothing to configure without a domain.

• **How to verify:** Start the Admin Server and connect with WLST (`connect()`); confirm the managed server(s) reach **RUNNING** and the C2M application is targeted to them.

• **How to implement if missing:** Complete the WebLogic/OUAF domain install and configuration (the *WebLogic 14c C2M Install & Configure* solution) before starting here.

2. Managed-server / cluster topology decision (single vs. clustered)

• **Description:** A decided topology — a single managed server, or a WebLogic **cluster** of managed servers — and, for clustered deployments, the intended member list.

• **Why required:** Coherence cluster association, JMS targeting (single server vs. cluster via a subdeployment), and IWS deployment targets all depend on whether you are clustered.

• **How to verify:** In the Administration Console under *Environment → Clusters* / *Servers*, confirm the intended servers/cluster exist and their membership.

• **How to implement if missing:** Define the cluster and assign managed servers before configuring Coherence/JMS targeting.

3. Network & ports for Coherence discovery

• **Description:** Network reachability between managed servers on the Coherence cluster listen

address/port (unicast **Well Known Addresses (WKA)** are strongly preferred; the default Coherence cluster listen port is **7574** — *verify for your version/topology*), with any host firewalls open for that port and, if multicast is used, multicast enabled on the segment.

• **Why required:** Coherence members discover and join the cluster over this transport; blocked ports or disabled multicast are the most common reason a cluster "won't form".

• **How to verify:** From each host, confirm the listen port is open to the other members (for example a TCP reachability check to the WKA hosts:port); confirm the chosen transport (unicast WKA vs. multicast) is permitted on the segment.

• **How to implement if missing:** Open the port between members and standardise on **unicast WKA** (deterministic, firewall-friendly) rather than multicast for most enterprise networks.

4. Persistent store location for JMS (file or JDBC)

• **Description:** A decided persistence backing for the JMS server — a **file store** on a path reachable by the target server(s), or a **JDBC store** in the database — so persistent messages survive restarts.

• **Why required:** A JMS server needs a persistent store for persistent/durable messaging; a missing or unreachable store prevents the JMS server from starting or loses messages.

• **How to verify:** Confirm the file-store directory exists and is writable by the WebLogic OS account, or the JDBC store's data source and schema exist.

• **How to implement if missing:** Create the file-store directory (owned by the product OS account) or the JDBC store / data source before creating the JMS server.

5. C2M message senders / JMS integration requirements defined

• **Description:** The list of JMS destinations C2M actually needs — queues/topics for outbound message senders, real-time hand-offs, and any JMS consumers — with their intended **JNDI names**.

• **Why required:** Creating arbitrary queues is not the goal; the destinations must match what the C2M **Message Sender** / integration configuration references, or messages will not flow.

• **How to verify:** In C2M, review the **Message Sender** configuration and any JMS-based integration to list the JNDI names expected; reconcile against the JMS module you will create.

• **How to implement if missing:** Agree the destination list and JNDI naming with the integration team before creating JMS resources.

6. IWS enabled and the web-services EAR deployable

• **Description:** The OUAF environment configured for **Inbound Web Services** (the web-services

functionality enabled via `configureEnv`), with the ability to run `initialSetup` and deploy the `spl-web-services` EAR, plus the WebLogic user-config/key files (`$SPLEBASE/etc/.wlsuserconfig`, `.wlsuserkey`) for headless deployment.

• **Why required:** IWS is deployed as a Java EE web application through the OUAF `iwsdeploy` utility; it will not deploy unless the environment is configured for web services and the Admin Server is up.

• **How to verify:** Confirm the IWS parameters are set in `configureEnv` (the *IWS Deployment Target* and *Web Admin Server Host*), that `$SPLEBASE/etc/.wlsuserconfig` / `.wlsuserkey` exist, and that `iwsdeploy` is present under `$SPLEBASE/bin`.

• **How to implement if missing:** Run `configureEnv` to enable web services and set the IWS parameters, create the WebLogic user-config/key files with `storeUserConfig`, then `initialSetup`, before deploying. See MOS **Doc ID 1644914.1** for XAI → IWS migration where legacy XAI services are still in use.

7. Inventory of inbound services to enable

• **Description:** The specific OUAF **Inbound Web Service** metadata records that must be *active/enabled* for the integrations in scope (the annotated services that IWS exposes).

• **Why required:** Deploying the EAR makes the container available, but each inbound service must be defined and enabled to actually respond; enabling everything expands the attack/support surface needlessly.

• **How to verify:** In C2M, list the **Inbound Web Service** records and their status; confirm each in-scope service exists and note its endpoint/operation.

• **How to implement if missing:** Define/enable the required Inbound Web Service records (and redeploy so the EAR reflects them) — enabling only what integrations need.

8. Backups and a rollback point before changes

• **Description:** A captured baseline — domain `config.xml` (and JMS/Coherence config), the $SPLEBASE product/config, and any JMS persistent-store state — plus a change window for production.

• **Why required:** Coherence, JMS, and deployment changes are domain-level and high-impact; a clean restore path is the rollback.

• **How to verify:** Confirm a current backup of the domain config and product home, and that you can restore them.

• **How to implement if missing:** Take the backups and secure the change approval before proceeding (per the global change-control requirement).

Gate to begin implementation: all eight items above pass on the **pilot (non-production)** environment, the object names are agreed (Overview naming convention), and a rollback is in hand.

Global Requirements — Oracle Utilities C2M / WebLogic Solutions

These requirements apply to **every** solution in the **Oracle** category (Oracle Utilities Customer to Meter and its Oracle WebLogic Server middleware). Verify each before starting a solution's steps.

> ***Vendor & version boundary (applies to every Oracle solution).*** *Oracle Utilities* ***Customer to Meter (C2M)****,* ***Customer Care & Billing (CC&B)****,* ***Meter Data Management (MDM)****, and* ***Oracle WebLogic Server*** *are authored from Oracle's official product documentation and the Oracle Utilities Application Framework (OUAF) install/admin guides.* ***Version-specific details*** *(C2M 25.4 / minimum v2.9, WebLogic 14c) are* ***cited or flagged "verify for your version"*** *— never invented. Some Oracle documentation and patches are entitlement-gated on* ***My Oracle Support (MOS)****; where a step requires a MOS note, patch number, or download, the solution points to it rather than reproducing gated content. All work assumes a* ***non-production environment first*** *and change control for production.*

1. Licensed Oracle Utilities C2M + WebLogic entitlement

• **Description:** A licensed Oracle Utilities **Customer to Meter** (and the underlying OUAF) plus the supported **Oracle WebLogic Server 14c** and database (Oracle Database) at the versions in scope.

• **Why required:** Installation media, patches, and support all flow from the entitlement.

• **How to verify:** Confirm the license and access to **My Oracle Support**; check the certified version matrix for C2M ↔ WebLogic ↔ Database ↔ Java.

• **How to implement if missing:** Obtain licensing/entitlement and MOS access before proceeding.

2. Certified platform stack (OS, Java, Database, WebLogic)

• **Description:** A host OS, JDK, Oracle Database, and WebLogic version combination **certified** for the target C2M release (per Oracle's certification/compatibility matrix).

• **Why required:** C2M/OUAF only supports certified combinations; uncertified stacks cause install and patch failures and lose supportability.

• **How to verify:** Compare the environment against the C2M release's certification matrix (MOS).

• **How to implement if missing:** Align OS/JDK/DB/WebLogic to a certified combination first.

3. Environment topology (Dev / Test / Prod) & access

• **Description:** Defined **Development, Test, and Production** environments (and any QA/UAT), with OS accounts (e.g. the OUAF/`splunk`-style product user), WebLogic admin credentials, and DB schema access appropriate to the task.

• **Why required:** C2M is deployed and promoted across environments; each solution assumes the target

environment exists and you can reach WebLogic, the file system, and the database.

• **How to verify:** Confirm you can log in to the OS product account, the WebLogic Administration Console / WLST, and the C2M database schema.

• **How to implement if missing:** Stand up / obtain access to the environment before starting.

4. Administrative access & tooling (WLST, shell, SQL)

• **Description:** Administrative access and the standard toolchain: **WLST (Jython)** for WebLogic, the OS **shell** (bash/ksh) for OUAF utilities (e.g. `splmanager`, `configureEnv`, `initialSetup`, batch/threadpool scripts), and **SQL*Plus / SQLcl** for the database.

• **Why required:** C2M/WebLogic administration is performed through these interfaces, not a GUI-only workflow; the solutions' automation is authored as WLST/shell/SQL.

• **How to verify:** Confirm WLST connects to the domain, shell access to the product home, and SQL connectivity to the schema.

• **How to implement if missing:** Provision the access and install the client tooling.

5. Change control, backup & a non-production pilot

• **Description:** A **change-control** process for production changes, verified **backups** (domain config, OUAF product/config, database) before disruptive actions (install, patch, upgrade, migration, tuning), and a **non-production environment** to validate on first.

• **Why required:** Every solution follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** with a documented rollback; C2M patches/upgrades and WebLogic changes are high-impact.

• **How to verify:** Confirm a backup/restore path and a change window/approval for production.

• **How to implement if missing:** Establish backups and change control before touching production.

6. Collaborating teams & support model

• **Description:** Engagement with the **infrastructure, DBA, network, application-development, and release/test **teams, and (post go-live) the** hypercare / on-call** support model and monitoring stack (e.g. Dynatrace) the environment uses.

• **Why required:** C2M administration is cross-functional; issues span middleware, database, network, and application layers, and production support runs on a defined escalation and monitoring model.

• **How to verify:** Confirm the named contacts, escalation path, and monitoring tooling.

• **How to implement if missing:** Identify the teams and the support/monitoring model before go-live.

*The deployment model across every Oracle solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **WLST (Jython), shell, and SQL** scripts — not PowerShell.*

Implementation Guide — Configuring the C2M Runtime (Coherence · JMS · IWS)

This guide configures the C2M runtime layer end to end: the **Coherence cluster**, the **JMS server / module / queues / topics / connection factory**, and **Inbound Web Services (IWS)** — then validates clustering, messaging, and web-service calls. It follows **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, on the **pilot (non-production)** environment first, with pass/fail gates and a rollback.

Object names below match the Overview naming convention and are reused throughout; substitute your environment's standard names but keep them consistent across console, WLST, and C2M config.

> ***Reference names:*** *cluster* `c2m-coherence-cluster` *(listen port 7574, *verify for your version*); JMS server* `C2MJMSServer`*; JMS system module* `C2MJMSModule`*; connection factory* `C2MConnectionFactory` *(JNDI* `jms/C2MConnectionFactory`*); queue* `C2MRealTimeQueue` *(JNDI* `jms/C2MRealTimeQueue`*); topic* `C2MEventTopic` *(JNDI* `jms/C2MEventTopic`*); subdeployment* `C2MJMSSubDeployment`*; IWS EAR* `spl-web-services`*.*
>
> ***Safety rules for every phase.*** *Take a config backup first (Requirement 8). Coherence and many JMS changes require a **server restart** to take effect — plan the restart into the change window. Do disruptive work in a maintenance window; keep the rollback (restore* `config.xml` */ product home) ready.*

Phase 0 — Assess (read-only)

Goal: capture the current runtime state before changing anything.

1. Connect WLST read-only and record: managed-server states, any existing Coherence cluster and its

members, existing JMS servers/modules/destinations, and whether `spl-web-services` is already deployed.

1. Run `OC2MRUNTIME-Discovery` (read-only) to snapshot Coherence, JMS, and IWS configuration to a

file, and `OC2MRUNTIME-ScopeHelper` to fix the exact target set (which servers/cluster you will touch) so later scripts act only in scope.

1. Confirm all Requirements pass on the pilot environment.

Gate 0 (pass/fail): you have a written baseline, an agreed target scope, and a rollback point. Do not proceed until all pass.

Phase 1 — Configure the Coherence cluster

Goal: a named Coherence cluster the managed servers reliably join.

1. **Create the Coherence cluster.** In the Administration Console: *Environment → Coherence Clusters

→ New*. Name it `c2m-coherence-cluster`. Choose **Configure within WebLogic Server**, select the transport (**unicast** — preferred — or multicast), and set the **cluster listen port** (default **7574**, *verify for your version*). (Console path per Oracle's *Create Coherence Clusters* task help.)

Equivalent WLST (offline or online + `startEdit()`), illustrative:

```
edit(); startEdit()
cd('/')
cmo.createCoherenceClusterSystemResource('c2m-coherence-cluster')
cd('/CoherenceClusterSystemResources/c2m-coherence-cluster/CoherenceClusterResource/'
   'c2m-coherence-cluster/CoherenceClusterParams/c2m-coherence-cluster')
cmo.setClusteringMode('unicast')        # unicast WKA preferred over multicast
cmo.setUnicastListenPort(7574)          # verify for your version/topology
save(); activate()
```

1. **Associate the cluster with the WebLogic server(s)/cluster.** Target

`c2m-coherence-cluster` to the managed server(s) or the WebLogic cluster that runs C2M so they become Coherence members. For **unicast**, configure the **Well Known Addresses (WKA)** to the member hosts (deterministic discovery; firewall-friendly) rather than relying on multicast.

1. **Restart** the affected managed server(s). *All Coherence changes require a restart to take

effect.*

1. **Verify formation** with `OC2MRUNTIME-Validation` (read-only): confirm the cluster is

present, every in-scope server is a **member**, and the member count equals the expected node count.

Gate 1: cluster `c2m-coherence-cluster` exists, all in-scope servers are members, no discovery errors in the server logs. If members are missing → check the listen port/firewall and unicast WKA before proceeding.

Phase 2 — Configure JMS (server → module → destinations → connection factory)

Goal: JMS resources C2M's message senders/integration can bind to and that survive restart.

1. **Create the JMS server + persistent store.** Create a **persistent store** (file store on a

writable path owned by the product OS account, or a JDBC store), then a **JMS server** `C2MJMSServer` using that store, targeted to the managed server (single-server) — for a cluster you will target destinations via the subdeployment in step 5.

1. **Create the JMS system module.** *Services → Messaging → JMS Modules → New* →

`C2MJMSModule`, targeted to the managed server or cluster running C2M.

1. **Create the connection factory.** In `C2MJMSModule`, create `C2MConnectionFactory` with JNDI

name `jms/C2MConnectionFactory`. (This is the CF the C2M message senders / JMS consumers reference.)

1. **Create the destinations.** In `C2MJMSModule`, create the queue `C2MRealTimeQueue` (JNDI

`jms/C2MRealTimeQueue`) and the topic `C2MEventTopic` (JNDI `jms/C2MEventTopic`). For a **clustered** deployment prefer **Uniform Distributed** destinations so every cluster member has a member destination.

1. **Create the subdeployment and target it.** Create subdeployment `C2MJMSSubDeployment` and

target the connection factory and destinations to `C2MJMSServer` (single server) or to the cluster's JMS servers. A **subdeployment** is the mechanism that groups module resources and targets them to a JMS

server / server / cluster — use it rather than the module default targets so targeting is explicit.

Illustrative WLST (online, inside `startEdit()`), matching Oracle's *Using WLST to Manage JMS* examples:

```
edit(); startEdit()
cd('/JMSSystemResources/C2MJMSModule/JMSResource/C2MJMSModule')

cf = cmo.createConnectionFactory('C2MConnectionFactory')
cf.setJNDIName('jms/C2MConnectionFactory')
cf.setSubDeploymentName('C2MJMSSubDeployment')

q = cmo.createQueue('C2MRealTimeQueue')
q.setJNDIName('jms/C2MRealTimeQueue')
q.setSubDeploymentName('C2MJMSSubDeployment')

t = cmo.createTopic('C2MEventTopic')
t.setJNDIName('jms/C2MEventTopic')
t.setSubDeploymentName('C2MJMSSubDeployment')

save(); activate()
```

1. **Reconcile with C2M.** Confirm the JNDI names created here match what the C2M **Message Sender**

configuration references (Requirement 5). A destination that does not match the sender's JNDI name will look healthy in WebLogic yet carry no traffic.

1. **Restart** if required and **verify** with `OC2MRUNTIME-Validation`: JMS server **RUNNING**,

destinations present and bound to JNDI, connection factory resolvable.

Gate 2: `C2MJMSServer` running, `C2MConnectionFactory` + `C2MRealTimeQueue` + `C2MEventTopic` present with the expected JNDI names, subdeployment targeting correct, and names reconciled with C2M message senders.

Phase 3 — Deploy & enable Inbound Web Services (IWS)

Goal: the `spl-web-services` EAR deployed and the in-scope inbound services responding.

1. **Enable web services in the environment.** From `$SPLEBASE/bin`, run `configureEnv` (`-a`) and set

the IWS parameters — the **IWS Deployment Target** (the server the EAR deploys to) and the **Web Admin Server Host** (the WebLogic Admin Server host). In non-production you may also set the development-environment flag to allow deploying from the browser UI; leave it **off** in production. Process the changes (P).

1. **Run initial setup.** From `$SPLEBASE/bin` run `initialSetup` so the templated config

(`weblogic.xml`, `web.xml`, `spl.properties`, etc.) is regenerated for IWS.

1. **Prepare headless WebLogic credentials.** Ensure `$SPLEBASE/etc/.wlsuserconfig` and

`.wlsuserkey` exist (created with the WLST `storeUserConfig` command) so the deploy runs without an interactive password. Import the WebLogic certificate into the OUAF trust store with `initialSetup -i` if TLS is in play.

1. **Ensure the Admin Server (and managed server) are running**, then **deploy the EAR** from

`$SPLEBASE/bin`:

```
cd $SPLEBASE/bin
ksh ./iwsdeploy.sh        # deploys the spl-web-services EAR to the IWS Deployment Target
```

(On Windows the equivalent is `iwsdeploy.cmd`.) The Admin Server **must be up** before running the deploy.

1. **Enable the required inbound services.** In C2M, activate the **Inbound Web Service** metadata

records that are in scope (Requirement 7) and redeploy if the service list changed. Enable **only** the services integrations need.

1. **Verify** with `OC2MRUNTIME-Validation`: the `spl-web-services` application is **deployed/active**

on the target server, and the in-scope inbound service endpoints respond (WSDL/endpoint reachable).

Gate 3: `spl-web-services` active on the IWS Deployment Target; each in-scope inbound service is enabled and returns a well-formed response (or WSDL) to a test call.

Phase 4 — Validate the runtime end to end (Pilot)

Goal: prove clustering + messaging + web services together on the pilot before expanding.

1. Run `OC2MRUNTIME-Validation` across all in-scope servers for a single consolidated pass on

Coherence membership, JMS destination health, and IWS deployment status.

1. Run `OC2MRUNTIME-Report` (read-only) to produce a health/coverage report (which servers are

Coherence members, which destinations exist and their consumer/pending counts, which inbound services are enabled and responding).

1. Execute the functional checks in **05-testing.md** (cluster formed, JMS message round-trips, an

inbound web-service call succeeds end to end).

1. If anything is off, correct with `OC2MRUNTIME-Remediation` (**dry-run by default** — review the

planned actions, then re-run with the explicit apply flag inside the change window).

Gate 4: the report shows all three layers healthy on the pilot and the functional tests pass.

Phase 5 — Expand to remaining environments

1. Repeat Phases 1–4 per environment (Test → Prod), each behind its own change window, backup, and

rollback.

1. Use `OC2MRUNTIME-ScopeHelper` to retarget the scripts to each environment's server set so you run

the **same** validated steps everywhere.

1. In production, keep the development-environment IWS flag **off**, enable only required inbound

services, and confirm the report is clean before closing the change.

Final gate: every target environment shows a clean `OC2MRUNTIME-Report` — Coherence formed, JMS flowing, IWS responding — with a captured rollback point.

Rollback

- **Coherence/JMS:** restore the previous domain `config.xml` (from the Requirement 8 backup) and restart; or remove the newly created cluster/JMS objects via WLST and restart.
- **IWS:** undeploy `spl-web-services` (or redeploy the prior EAR) and disable the newly enabled inbound services.
- Re-run `OC2MRUNTIME-Validation` to confirm the environment is back to its pre-change baseline.

Sources

- Oracle® Fusion Middleware — *Create Coherence Clusters* (task help) and *Configuring and Managing

Coherence Clusters*: https://docs.oracle.com/en/middleware/fusion-middleware/weblogic-server/12.2.1.4/wlach/taskhelp/coherence/CreateCoherenceClusters.html

- Oracle® Fusion Middleware — *Using WLST to Manage JMS Servers and JMS System Module Resources*

(WebLogic 14.1.1): https://docs.oracle.com/en/middleware/standalone/weblogic-server/14.1.1.0/jmsad/wlst.html

- Oracle Utilities CCB — *Deploying Inbound Web Services (IWS)*:

https://docs.oracle.com/en/industries/energy-water/ccb/29011/ccb-installation-guide/CCB-INSTALLATION-GUIDE/CCB_Install_Postinstall.09.09.html

- Oracle Utilities Application Framework — *Server Administration Guide* (Web Application Server

Configuration): https://docs.oracle.com/en/industries/energy-water/framework/254/server-admin-guide/OUAF_SERVER_ADMIN_254/OUAF_Server_Admin_Configuration.06.4.html

- My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Testing — Validating Coherence, JMS & IWS

These tests prove the three runtime layers work **individually and together**, on the pilot environment first. Each test states the **objective**, **steps**, and an explicit **pass criterion**. Run the read-only scripts (`OC2MRUNTIME-Discovery`, `OC2MRUNTIME-Validation`, `OC2MRUNTIME-Report`) for repeatable evidence; capture output as change-record proof.

> *Run tests on **non-production first**. A failing gate here is a stop — fix the cause (see 06-troubleshooting.md) before promoting.*

A. Coherence clustering

A1 — Cluster exists and is targeted

- **Objective:** the named cluster is defined and associated with the right servers.
- **Steps:** run `OC2MRUNTIME-Validation`; or in the console *Environment → Coherence Clusters* open

`c2m-coherence-cluster` and review its targets.

- **Pass:** `c2m-coherence-cluster` exists, transport/listen port match the design (default 7574,

verify for your version), and it is targeted to every in-scope server/cluster.

A2 — All nodes are members

- **Objective:** every in-scope managed server actually joined the cluster.
- **Steps:** with all in-scope managed servers **RUNNING**, run `OC2MRUNTIME-Validation` and read the

member list/count; corroborate with "member joined/left" entries in each server log.

- **Pass:** member count equals the expected node count; no missing members; no repeated

join/leave churn.

A3 — No discovery errors after restart

- **Objective:** the cluster forms cleanly on a cold start (changes need a restart).
- **Steps:** restart the in-scope servers; scan logs for Coherence discovery/timeout errors; re-run

`OC2MRUNTIME-Validation`.

- **Pass:** clean formation, no unicast WKA/multicast discovery errors, stable membership.

B. JMS messaging

B1 — JMS server and destinations present

- **Objective:** the JMS objects exist and are bound to JNDI.
- **Steps:** run `OC2MRUNTIME-Validation`; or console *Services → Messaging → JMS Modules →

C2MJMSModule*.

- **Pass:** `C2MJMSServer` is **RUNNING**; `C2MConnectionFactory` (`jms/C2MConnectionFactory`),

`C2MRealTimeQueue` (`jms/C2MRealTimeQueue`), `C2MEventTopic` (`jms/C2MEventTopic`) all present with the expected JNDI names and correct subdeployment targeting.

B2 — Message round-trip (produce → consume)

- **Objective:** a message actually flows through the queue.
- **Steps:** on `C2MRealTimeQueue`, note the current/pending counts (Monitoring tab or

`OC2MRUNTIME-Report`); send a test message (a controlled C2M message-sender test, or WebLogic's JMS test facility on the destination); observe the count increment then drain as it is consumed.

- **Pass:** the test message is enqueued and then consumed (pending returns to baseline); no messages

land in an error/pending-forever state.

B3 — JNDI names reconcile with C2M message senders

• **Objective:** what C2M sends to is what exists in WebLogic.

• **Steps:** compare the JNDI names in the C2M **Message Sender** config against the module's destinations (from `OC2MRUNTIME-Report`).

• **Pass:** every JNDI name the C2M senders reference resolves to a real destination — no orphans on either side.

B4 — Persistence survives restart

• **Objective:** persistent messages are not lost on restart.

• **Steps:** enqueue a persistent test message, do **not** consume it, restart the JMS server's host, re-check the queue.

• **Pass:** the persistent message is still present after restart (file/JDBC store intact).

C. Inbound Web Services (IWS)

C1 — EAR deployed and active

• **Objective:** the web-services application is live on the IWS Deployment Target.

• **Steps:** run `OC2MRUNTIME-Validation`; or console *Deployments*.

• **Pass:** `spl-web-services` is **Active/OK** on the intended target server.

C2 — Inbound service responds

• **Objective:** an enabled inbound service answers a call.

• **Steps:** retrieve the WSDL/endpoint for an in-scope Inbound Web Service; issue a well-formed test request (SOAP/REST as applicable) with valid credentials.

• **Pass:** the service returns a well-formed, expected response (or the WSDL loads); disabled/out-of-scope services are **not** exposed.

C3 — End-to-end integration path

• **Objective:** an inbound call drives real C2M behaviour, including any JMS hand-off.

• **Steps:** call an inbound service that triggers downstream work; confirm the C2M record/outcome and, where the flow hands off asynchronously, confirm the message appears on `C2MRealTimeQueue` and is consumed.

• **Pass:** the business outcome is correct **and** the asynchronous hand-off flows through JMS — proving the three layers work together.

D. Consolidated runtime health

D1 — Full runtime report is clean

• **Objective:** one artifact proving Coherence + JMS + IWS are healthy across scope.

• **Steps:** run `OC2MRUNTIME-Report` across the in-scope servers; review coverage (members present, destinations healthy, inbound services enabled/responding).

• **Pass:** report shows all three layers healthy with no gaps for any in-scope server — the pilot validation gate. Retain the report as change evidence.

Exit criteria for the pilot: A1–A3, B1–B4, C1–C3, and D1 all pass, with saved script output as evidence. Only then expand to the next environment.

Troubleshooting — Coherence, JMS & IWS

Common failures across the C2M runtime, as **symptom** → **likely cause** → **resolution**. Start from the read-only scripts (`OC2MRUNTIME-Discovery`, `OC2MRUNTIME-Validation`, `OC2MRUNTIME-Report`) to localise the problem before changing anything. All fixes are non-production-first with a rollback.

Coherence

Symptom: cluster won't form / a node never becomes a member.

• *Likely cause:* the Coherence listen port is blocked between hosts, or multicast is disabled on the segment.

• *Resolution:* confirm the listen port (default 7574, *verify for your version*) is open host-to-host; switch from multicast to **unicast WKA** listing the member hosts explicitly; restart and re-verify with `OC2MRUNTIME-Validation`.

Symptom: members join then drop (join/leave churn).

• *Likely cause:* network latency/packet loss, or clock skew between member hosts.

• *Resolution:* put members on a low-latency segment; enforce NTP so clocks agree; check logs for timeout entries and raise the relevant timeouts only if the network is known-good.

Symptom: changes don't take effect.

• *Likely cause:* Coherence changes require a server restart and it wasn't done.

• *Resolution:* restart the affected managed server(s); re-run `OC2MRUNTIME-Validation`.

Symptom: extra/wrong servers appear as members.

• *Likely cause:* the cluster is associated with more servers than intended.

• *Resolution:* correct the cluster targeting so only in-scope servers are members; restart affected nodes.

JMS

Symptom: JMS server won't start.

• *Likely cause:* the persistent store path is missing/not writable by the product OS account, or the JDBC store's data source is down.

• *Resolution:* create/repair the file-store directory (correct ownership) or fix the JDBC store/data source; restart the JMS server.

Symptom: a queue exists in WebLogic but no messages ever arrive.

• *Likely cause:* the destination's JNDI name does not match what the C2M **Message Sender** references.

• *Resolution:* reconcile JNDI names with `OC2MRUNTIME-Report`; align the C2M message-sender JNDI or the destination JNDI so they match exactly.

Symptom: messages pile up (pending count keeps rising).

• *Likely cause:* no consumer is attached, the consumer errors, or a quota/backlog limit is hit.

• *Resolution:* confirm the consumer (C2M/IWS) is running and error-free; check destination quota and error state; drain/redeliver once the consumer is healthy.

Symptom: persistent messages lost on restart.

• *Likely cause:* messages were non-persistent, or the persistent store was not used/was recreated.

• *Resolution:* ensure the connection factory/destination use persistent delivery and a stable file/JDBC store that is backed up.

Symptom: destinations don't appear on all cluster members.

• *Likely cause:* standalone (not Uniform Distributed) destinations, or subdeployment targeting only one server.

• *Resolution:* use **Uniform Distributed** destinations and target the subdeployment (`C2MJMSSubDeployment`) to the cluster's JMS servers; re-verify.

Inbound Web Services (IWS)

Symptom: `iwsdeploy` **fails.**

• *Likely cause:* the WebLogic Admin Server (or target managed server) is not running, or the stored user-config/key files are missing/wrong.

• *Resolution:* start the Admin Server and target server; recreate `$SPLEBASE/etc/.wlsuserconfig/.wlsuserkey` with `storeUserConfig`; re-run `iwsdeploy` from `$SPLEBASE/bin`.

Symptom: EAR deploys but inbound calls return 404 / service not found.

- *Likely cause:* the specific Inbound Web Service record isn't enabled, or the EAR wasn't regenerated after enabling it.
- *Resolution:* enable the required Inbound Web Service in C2M (Requirement 7); run `initialSetup` and redeploy so the EAR reflects the service list.

Symptom: web-services functionality unavailable / config missing.

- *Likely cause:* IWS not enabled in `configureEnv`, or `initialSetup` not run after enabling it.
- *Resolution:* run `configureEnv` to enable web services and set the **IWS Deployment Target** / **Web Admin Server Host**; run `initialSetup`; redeploy.

Symptom: inbound calls fail TLS/handshake.

- *Likely cause:* the WebLogic certificate isn't in the OUAF trust store.
- *Resolution:* import it with `initialSetup -i`; confirm the endpoint's TLS/WS-Policy matches the caller.

Symptom: browser-UI deploy works in Test but the option is missing in Prod.

- *Cause (expected):* the development-environment flag is intentionally **off** in production.
- *Resolution:* deploy via `iwsdeploy` from `$SPLEBASE/bin` — the correct production path; leave the flag off.

Symptom: legacy XAI inbound service no longer works.

- *Likely cause:* XAI has been superseded by IWS.
- *Resolution:* migrate the service to IWS per MOS **Doc ID 1644914.1**; define/enable the equivalent Inbound Web Service and redeploy.

Escalation

If Coherence membership, JMS binding, or IWS deployment still fails after the above, gather the `OC2MRUNTIME-Discovery` snapshot, `OC2MRUNTIME-Validation` output, and the relevant server logs, and raise a My Oracle Support case referencing the certified version matrix (C2M ↔ WebLogic ↔ DB ↔ Java) — some Coherence/IWS notes and patches are MOS-gated.

Sources

- Oracle® Fusion Middleware — *Configuring and Managing Coherence Clusters*: https://docs.oracle.com/en/middleware/fusion-middleware/weblogic-server/12.2.1.4/clust/coherence.html
- Oracle® Fusion Middleware — *Using WLST to Manage JMS Servers and JMS System Module Resources*: https://docs.oracle.com/en/middleware/standalone/weblogic-server/14.1.1.0/jmsad/wlst.html

• Oracle Utilities CCB — *Deploying Inbound Web Services (IWS)*:

https://docs.oracle.com/en/industries/energy-water/ccb/29011/ccb-installation-guide/CCB-INSTALLATION-GUIDE/CCB_Install_Postinstall.09.09.html

• My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Best Practices — Coherence, JMS & IWS on C2M

Production-safe practices for the C2M runtime layer. They complement the implementation steps and keep the three layers healthy over time.

Coherence

• **Prefer unicast Well Known Addresses (WKA) over multicast.** Unicast WKA is deterministic and firewall-friendly; multicast is fragile across enterprise segments and a leading cause of "cluster won't form". List the member hosts explicitly.

• **Standardise the cluster listen port and open it between all members.** Use one agreed port (default 7574, *verify for your version*) and confirm host firewalls allow it host-to-host.

• **One named cluster, associated to the right servers.** Keep a single, clearly named cluster (`c2m-coherence-cluster`) targeted only to the servers that must be members — don't leave stray associations.

• **Plan the restart.** All Coherence changes require a server restart; batch changes into one restart in the change window rather than repeated bounces.

• **Verify membership after every change and after every restart** with `OC2MRUNTIME-Validation` — member count must equal expected node count. Watch logs for join/leave churn (a sign of network or timeout issues).

• **Keep member clocks in sync (NTP)** and member hosts on a low-latency segment; clock skew and latency destabilise membership.

JMS

• **Always back destinations with a persistent store** (file store on a writable, backed-up path owned by the product account, or a JDBC store) so persistent messages survive restarts.

• **Target with an explicit subdeployment, not module defaults.** A named subdeployment (`C2MJMSSubDeployment`) makes targeting deliberate and auditable, and is required for clean cluster targeting.

• **Use Uniform Distributed destinations in a cluster** so every member has a member destination and no single node is a bottleneck or single point of failure.

• **Make JNDI names match C2M message senders exactly.** The most common "silent" JMS failure is a

healthy-looking destination whose JNDI name does not match what C2M sends to. Reconcile on every change (`OC2MRUNTIME-Report`).

• **Set quotas and monitor pending/error counts.** Configure destination/quota limits so a backlog is

bounded, and alert on rising pending counts or messages parked in error state.

• **Name resources consistently** (`C2MJMSServer` / `C2MJMSModule` / `C2MConnectionFactory` /

`C2MRealTimeQueue` / `C2MEventTopic`) across environments so the same scripts and runbooks apply everywhere.

Inbound Web Services (IWS)

• **Migrate off XAI to IWS** where legacy XAI inbound services remain; IWS uses the container's

clustering/HA and per-service monitoring. See MOS Doc ID 1644914.1 for the migration path.

• **Deploy headless and repeatably.** Use `iwsdeploy` with the stored WebLogic user-config/key files

(`$SPLEBASE/etc/.wlsuserconfig` / `.wlsuserkey`) so deploys are scriptable and identical across environments — no interactive passwords.

• **Turn the development-environment flag OFF in production.** The "deploy from browser UI" option is

a non-production convenience; leaving it on in production is an unnecessary exposure.

• **Enable only the inbound services integrations actually need.** Every enabled service is surface

area to secure, monitor, and support; enable by inventory (Requirement 7), not by default.

• **Confirm the Admin Server is up before deploying.** `iwsdeploy` requires the Admin Server (and the

target managed server) running; a stopped Admin Server is the classic deploy failure.

• **Secure the endpoints** (TLS via the OUAF trust store / `initialSetup -i`, plus WS-Policy /

authentication as required) and monitor each service — with Enterprise Manager if available — for latency and errors.

Cross-cutting

• **Non-production first, always.** Configure and validate on the pilot, then promote the *same*

named objects and *same* scripts to Test and Prod.

• **Back up before disruptive changes** (domain `config.xml`, `$SPLEBASE`, JMS store) and keep the

restore path as the rollback.

• **Reuse names across environments** so discovery/validation/report scripts are portable — retarget

with `OC2MRUNTIME-ScopeHelper` rather than re-authoring per environment.

• **Keep remediation dry-run by default.** Review `OC2MRUNTIME-Remediation`'s planned actions before

applying, and apply only inside a change window.

• **Retain script output as evidence** for change records and audits; a clean `OC2MRUNTIME-Report` is the sign-off artifact.

Sources

• Oracle® Fusion Middleware — *Configuring and Managing Coherence Clusters*:

https://docs.oracle.com/en/middleware/fusion-middleware/weblogic-server/12.2.1.4/clust/coherence.html

• Oracle® Fusion Middleware — *Using WLST to Manage JMS Servers and JMS System Module Resources*:

https://docs.oracle.com/en/middleware/standalone/weblogic-server/14.1.1.0/jmsad/wlst.html

• Oracle Utilities CCB — *Deploying Inbound Web Services (IWS)*:

https://docs.oracle.com/en/industries/energy-water/ccb/29011/ccb-installation-guide/CCB-INSTALLATION-GUIDE/CCB_Install_Postinstall.09.09.html

• My Oracle Support — *Migrating from XAI to IWS*, Doc ID 1644914.1 (entitlement-gated).

Deliverables Summary — C2M Coherence, JMS & IWS Configuration

What this solution includes and how the pieces fit together.

Documentation

Document	Purpose
Solution Overview (`01`)	The C2M runtime — Coherence clustering, JMS messaging, Inbound Web Services — and how they support batch, real-time, and integration.
Requirements Guide (`02`)	Every prerequisite with how to *verify* and *implement* (domain, topology, Coherence network/ports, JMS store, C2M message-sender reconciliation, IWS enablement).
Implementation Guide (`03`)	Exact steps: Coherence cluster → JMS server/module/destinations/CF → IWS deploy/enable → validate, pilot-first, with pass/fail gates and rollback.
Testing Guide (`05`)	Objective/steps/pass-criterion tests for clustering, messaging, and web-service calls, plus a consolidated health gate.
Best Practices (`07`)	Production-safe guidance across Coherence, JMS, and IWS.
Troubleshooting Guide (`06`)	Symptom → likely cause → resolution for the common runtime failures.

Document	Purpose
Executive Summary	One-page business framing for sponsors.
Global Requirements	The Oracle-category prerequisites shared by every C2M/WebLogic solution.

Automation scripts (`scripts/`)

Five scripts authored in the authentic C2M/WebLogic toolchain — **WLST (Jython), shell, and SQL** (never PowerShell). Read-only scripts are safe to run anywhere; the remediation script is **dry-run by default**.

Script	Type	Mode	What it does
`OC2MRUNTIME-Discovery`	WLST	Read-only	Snapshots the current Coherence cluster(s), JMS servers/modules/destinations/CF, and IWS deployment to an evidence file.
`OC2MRUNTIME-Validation`	WLST	Read-only	Per-node pass/fail checks: Coherence membership, JMS destination health, `spl-web-services` deployment status.
`OC2MRUNTIME-Report`	WLST	Read-only	Health/coverage report across scope — members present, destination pending/consumer counts, inbound services enabled/responding.
`OC2MRUNTIME-Remediation`	WLST	**Dry-run default**	Plans/creates missing runtime objects (cluster association, JMS destinations/CF/subdeployment targeting) — prints the plan unless an explicit apply flag is set.
`OC2MRUNTIME-ScopeHelper`	shell	Helper	Resolves and validates the target scope (servers/cluster, connection details) and exports it so the other scripts act only in scope, portably across environments.

A small SQL helper query is included inline within the report tooling notes for reconciling C2M Inbound Web Service / Message Sender metadata where a database read is the fastest cross-check.

How they work together

`ScopeHelper` fixes the target set → `Discovery` captures the baseline → `Validation` gates each phase → `Remediation` (dry-run first) fills gaps → `Report` produces the sign-off artifact. Reuse the same named objects and rerun the same scripts in each environment (retargeted via `ScopeHelper`) so Test and Prod get the identical, validated configuration.

Outcome

The C2M runtime wired correctly — Coherence clustering, JMS messaging, and IWS integration all healthy — proven on a pilot before scaling, with saved evidence for change control.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

OC2MRUNTIME-Discovery.py

Purpose: Connect to a running WebLogic domain and snapshot the current C2M runtime configuration: Coherence cluster(s) and members, JMS servers / modules / destinations / connection factories, and the spl-web-services (IWS) deployment. Read-only baseline evidence before any change. Set admin-url, admin-user, and admin-password for your-domain via the environment variables below. HOW TO RUN Set OC2M_ADMIN_URL, OC2M_ADMIN_USER, OC2M_ADMIN_PASS (or source ScopeHelper output), then: java weblogic.WLST OC2MRUNTIME-Discovery.py Optionally: java weblogic.WLST OC2MRUNTIME-Discovery.py > oc2m-discovery.out EXPECTED RESULTS Prints a sectioned snapshot (COHERENCE / JMS / IWS) to stdout. Exit 0 on a clean read; nonzero only if it cannot connect. It never edits — safe to run any time. NOTES White-label, repeatable. Read-only: no edit()/startEdit() — it only reads the MBean tree, so it is safe against production. Reuse the same object names across environments.

How to run: Set OC2M_ADMIN_URL, OC2M_ADMIN_USER, OC2M_ADMIN_PASS (or source ScopeHelper output), then: java weblogic.WLST OC2MRUNTIME-Discovery.py Optionally: java weblogic.WLST OC2MRUNTIME-Discovery.py > oc2m-discovery.out EXPECTED RESULTS Prints a sectioned snapshot (COHERENCE / JMS / IWS) to stdout. Exit 0 on a clean read; nonzero only if it cannot connect. It never edits — safe to run any time. NOTES White-label, repeatable. Read-only: no edit()/startEdit() — it only reads the MBean tree, so it is safe against production. Reuse the same object names across environments.

OC2MRUNTIME-Remediation.py

Purpose: Bring the JMS/Coherence configuration up to the expected named objects: create the Coherence cluster if absent, the JMS server, the JMS module, the connection factory, the queue and topic, and the subdeployment, using the solution reference names. By default it only PRINTS the plan; it changes nothing until you pass the explicit apply flag. IWS deployment itself is done with the OUAF iwsdeploy utility (shell), not here. HOW TO RUN Dry-run (default, safe): java weblogic.WLST OC2MRUNTIME-Remediation.py Apply for real (inside a change window, after review): OC2M_APPLY=true java weblogic.WLST OC2MRUNTIME-Remediation.py Names/targets come from environment variables (see below); source ScopeHelper first. EXPECTED RESULTS Dry-run prints "[PLAN] would create ..." lines and makes no changes (exit 0). Apply mode prints "[APPLY] created ..." inside a single edit session and activates once; objects that already exist are reported "[SKIP] exists" and left untouched. NOTES White-label, repeatable. DRY-RUN is the default — it only edits when OC2M_APPLY=true, and even then only creates missing objects (idempotent), never deletes. A server restart is still required for Coherence/JMS changes to take effect.

How to run: Dry-run (default, safe): java weblogic.WLST OC2MRUNTIME-Remediation.py Apply for real (inside a change window, after review): OC2M_APPLY=true java weblogic.WLST OC2MRUNTIME-Remediation.py Names/targets come from environment variables (see below); source ScopeHelper first. EXPECTED RESULTS Dry-run prints "[PLAN] would create ..." lines and makes no changes (exit 0). Apply mode prints "[APPLY] created ..." inside a single edit session and activates once; objects that already exist are reported "[SKIP] exists" and left untouched. NOTES White-label, repeatable. DRY-RUN is the default — it only edits when OC2M_APPLY=true, and even then only creates missing

objects (idempotent), never deletes. A server restart is still required for Coherence/JMS changes to take effect.

OC2MRUNTIME-Report.py

Purpose: Produce a single human-readable coverage report across scope: which servers are Coherence members, each JMS destination with its current/pending/consumer counts, and which IWS application(s) are active. Intended as the pilot sign-off artifact and change-record evidence. Reads runtime metrics; makes no changes. Point it at your-domain via the environment variables below. HOW TO RUN Set OC2M_ADMIN_URL / OC2M_ADMIN_USER / OC2M_ADMIN_PASS, then: java weblogic.WLST OC2MRUNTIME-Report.py > oc2m-runtime-report.txt EXPECTED RESULTS A sectioned report (COHERENCE / JMS METRICS / IWS) with per-object counts and an overall coverage line. Exit 0 on a clean read; nonzero only if it cannot connect. NOTES White-label, repeatable. Read-only: uses domainRuntime() metric MBeans only, never edits — safe on production. For a database-side cross-check of C2M Inbound Web Service / Message Sender metadata, see the reconciliation query in the deliverables notes.

How to run: Set OC2M_ADMIN_URL / OC2M_ADMIN_USER / OC2M_ADMIN_PASS, then: java weblogic.WLST OC2MRUNTIME-Report.py > oc2m-runtime-report.txt EXPECTED RESULTS A sectioned report (COHERENCE / JMS METRICS / IWS) with per-object counts and an overall coverage line. Exit 0 on a clean read; nonzero only if it cannot connect. NOTES White-label, repeatable. Read-only: uses domainRuntime() metric MBeans only, never edits — safe on production. For a database-side cross-check of C2M Inbound Web Service / Message Sender metadata, see the reconciliation query in the deliverables notes.

OC2MRUNTIME-ScopeHelper.sh

Purpose: Centralise the connection details, expected object names, and in-scope server list so the WLST scripts (Discovery, Validation, Report, Remediation) act only where intended and use identical names across environments. Reads values from your environment or a small config file, validates them, and prints export lines you can source. Reuses the solution reference names by default; override per environment (your-env-config). HOW TO RUN Print the resolved scope: sh ./OC2MRUNTIME-ScopeHelper.sh Load it into your shell: . ./OC2MRUNTIME-ScopeHelper.sh --export With a config file: sh ./OC2MRUNTIME-ScopeHelper.sh --config myenv.conf Then run e.g.: java weblogic.WLST OC2MRUNTIME-Validation.py EXPECTED RESULTS Prints the resolved OC2M_* variables and a PASS/FAIL readiness check (admin URL set, scope servers set, iwsdeploy present). Exit 0 when ready to run the WLST scripts; exit 1 if a required value is missing so a pipeline stops before acting out of scope. NOTES White-label, repeatable. Helper only: it neither connects nor changes anything — it just resolves scope and env vars. Set OC2M_APPLY only when you intend Remediation to write; it defaults to false so the runtime stays dry-run safe.

How to run: Print the resolved scope: sh ./OC2MRUNTIME-ScopeHelper.sh Load it into your shell: . ./OC2MRUNTIME-ScopeHelper.sh --export With a config file: sh ./OC2MRUNTIME-ScopeHelper.sh --config myenv.conf Then run e.g.: java weblogic.WLST OC2MRUNTIME-Validation.py EXPECTED RESULTS Prints the resolved OC2M_* variables and a PASS/FAIL readiness check (admin URL set, scope servers set, iwsdeploy present). Exit 0 when ready to run the WLST scripts; exit 1 if a required value is missing so a pipeline stops before acting out of scope. NOTES White-label, repeatable. Helper only: it neither connects nor changes anything — it just resolves scope and env vars. Set OC2M_APPLY only when you intend Remediation to write; it defaults to false so the runtime stays dry-run safe.

OC2MRUNTIME-Validation.py

Purpose: Validate the three runtime layers against the expected named objects and print a PASS/FAIL line per check: Coherence cluster present and all in-scope servers are members; JMS server running with the expected destinations / connection factory; the spl-web-services (IWS) application deployed and active. Expected names default to the solution reference names and can be overridden by environment variables. HOW TO RUN Set OC2M_ADMIN_URL / OC2M_ADMIN_USER / OC2M_ADMIN_PASS (and optional OC2M_EXPECT_* / OC2M_SCOPE_SERVERS from ScopeHelper), then: java weblogic.WLST OC2MRUNTIME-Validation.py EXPECTED RESULTS One [PASS]/[FAIL] line per check and a summary count. Process exit code is 0 when every check passes, 1 when any check fails (so it gates a pipeline/change step). NOTES White-label, repeatable. Read-only: reads the config/runtime MBeans only, no edits — safe on production. Reuse the same expected names in every environment via ScopeHelper.

How to run: Set OC2M_ADMIN_URL / OC2M_ADMIN_USER / OC2M_ADMIN_PASS (and optional OC2M_EXPECT_* / OC2M_SCOPE_SERVERS from ScopeHelper), then: java weblogic.WLST OC2MRUNTIME-Validation.py EXPECTED RESULTS One [PASS]/[FAIL] line per check and a summary count. Process exit code is 0 when every check passes, 1 when any check fails (so it gates a pipeline/change step). NOTES White-label, repeatable. Read-only: reads the config/runtime MBeans only, no edits — safe on production. Reuse the same expected names in every environment via ScopeHelper.